Cyberscope

A TAC Security Company

Audit Report

Amplee

October 2026

Network BSC

Address 0xEd8998171E88B279b4258882F4a900FC50a752CF

Audited by © cyberscope

Analysis

Critical Medium Minor / Informative Pass

Severity	Code	Description	Status
●	ST	Stops Transactions	Passed
●	OTUT	Transfers User's Tokens	Passed
●	ELFM	Exceeds Fees Limit	Passed
●	MT	Mints Tokens	Passed
●	BT	Burns Tokens	Passed
●	BC	Blacklists Addresses	Passed

Diagnostics

Critical Medium Minor / Informative

Severity	Code	Description	Status
●	MZAV	Missing Zero Address Validation	Unresolved
●	ILI	Invalid License Identifier	Unresolved
●	MEM	Missing Error Messages	Unresolved
●	MEE	Missing Events Emission	Unresolved
●	L02	State Variables could be Declared Constant	Unresolved

Table of Contents

Analysis 1
Diagnostics 2
Table of Contents 3
Risk Classification 4
Review 5
Audit Updates 5
Source Files 5
Findings Breakdown 6
MZAV - Missing Zero Address Validation 7
Description 7
Recommendation 7
ILI - Invalid License Identifier 8
Description 8
Recommendation 8
MEM - Missing Error Messages 9
Description 9
Recommendation 9
MEE - Missing Events Emission 10
Description 10
Recommendation 10
L02 - State Variables could be Declared Constant 11
Description 11
Recommendation 11
Functions Analysis 12
Inheritance Graph 13
Flow Graph 14
Summary 15
Disclaimer 16
About Cyberscope 17

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation**: This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation**: This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical**: Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium**: Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor**: Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative**: Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
Critical	Highly Likely / High Impact
Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
Minor / Informative	Unlikely / Low to no Impact

Review

Contract Name	AMPLEE
Compiler Version	v0.8.34+commit.80d5c536
Optimization	200 runs
Explorer	https://bscscan.com/address/0xed8998171e88b279b4258882f4a900fc50a752cf
Address	0xed8998171e88b279b4258882f4a900fc50a752cf
Network	BSC
Symbol	AMPLEE
Decimals	18
Total Supply	500,000,000

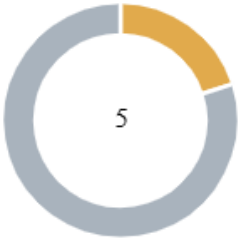
Audit Updates

Initial Audit	09 Oct 2026

Source Files

Filename	SHA256
contracts/3_Ballot1.sol	6d1721281bc030d0777bfb221e332fc37c9d53d7a6b33a5f85672073d180bbdd

Findings Breakdown



Critical	0
Medium	1
Minor / Informative	4

Severity	Unresolved	Acknowledged	Resolved	Other
Critical	0	0	0	0
Medium	1	0	0	0
Minor / Informative	4	0	0	0

MZAV - Missing Zero Address Validation

Criticality	Medium
Location	contracts/3_Ballot1.sol#L51,108
Status	Unresolved

Description

Neither `transfer()` or `transferFrom()` validates the recipient address. Holders can send tokens to the zero address, to the token contract itself, or to the `0x000000000000000000000000000000000000dEaD` address. The contract has no function that can move tokens out of these addresses, so any amount sent there is permanently lost.

Transfers to the zero address and the dead address are treated as burns by explorers and supply trackers, the same as the output of `burn()`. As a result, any holder can reduce the reported circulating supply through a plain transfer, while `totalSupply` continues to count all of these tokens as part of the supply.

```
require(balanceOf[msg.sender] >= _value);
balanceOf[msg.sender] -= _value;
balanceOf[_to] += _value;
emit Transfer(msg.sender, _to, _value);
```

Recommendation

The team is advised to validate the recipient of every transfer and reject addresses that cannot use the tokens. If burns are intended, they should only happen through the burn() function. This will prevent accidental token losses and keep burn activity traceable to a single entry point.

ILI - Invalid License Identifier

Criticality	Minor / Informative
Location	contracts/3_Ballot1.sol#L1
Status	Unresolved

Description

The license header reads UNLISCENSED, which is a misspelling of UNLICENSED and is not a valid SPDX identifier. The compiler accepts the value without a warning, so the mistake passes every build unnoticed. Block explorers and license scanners that parse the header may treat the source as having an unknown license.

```
// SPDX-License-Identifier: UNLISCENSED
```

Recommendation

The team is advised to use a valid SPDX license identifier in the source header. This will ensure that explorers and tooling report the license of the verified source correctly.

MEM - Missing Error Messages

Criticality	Minor / Informative
Location	contracts/3_Ballot1.sol#L49,61,105,106
Status	Unresolved

Description

The contract is missing error messages. Specifically, there are no error messages to accurately reflect the problem, making it difficult to identify and fix the issue. As a result, the users will not be able to find the root cause of the error.

```
require(balanceOf[msg.sender] >= _value);
...
require(_value <= balanceOf[_from]);
require(_value <= allowance[_from][msg.sender]);
```

Recommendation

The team is suggested to provide a descriptive message to the errors. This message can be used to provide additional context about the error that occurred or to explain why the contract execution was halted. This can be useful for debugging and for providing more information to users that interact with the contract.

MEE - Missing Events Emission

Criticality	Minor / Informative
Location	contracts/3_Ballot1.sol#L35
Status	Unresolved

Description

The contract assigns the entire supply to the deployer's balance in the constructor instead of going through a minting routine. No Transfer event from the zero address is emitted for this allocation, although the ERC-20 standard recommends one for token creation. As a result, explorers and indexers that rebuild balances from Transfer events do not register the initial supply and may report incorrect balances for the deployer.

```
constructor() {
    balanceOf[msg.sender] = totalSupply;
}
```

Recommendation

The team is advised to ensure the initial token creation is reported through the standard Transfer event. This will keep event-based balance tracking consistent with on-chain balances.

L02 - State Variables could be Declared Constant

Criticality	Minor / Informative
Location	contracts/3_Ballot1.sol#L6,7,8,9
Status	Unresolved

Description

State variables can be declared as constant using the constant keyword. This means that the value of the state variable cannot be changed after it has been set. Additionally, the constant variables decrease gas consumption of the corresponding transaction.

```
string public name = "AMPLE ECOSYSTEM"
string public symbol = "AMPLEE"
uint256 public totalSupply =500000000000000000000000000
uint8 public decimals = 18
```

Recommendation

Constant state variables can be useful when the contract wants to ensure that the value of a state variable cannot be changed by any function in the contract. This can be useful for storing values that are important to the contract's behavior, such as the contract's address or the maximum number of times a certain function can be called. The team is advised to add the constant keyword to state variables that never change.

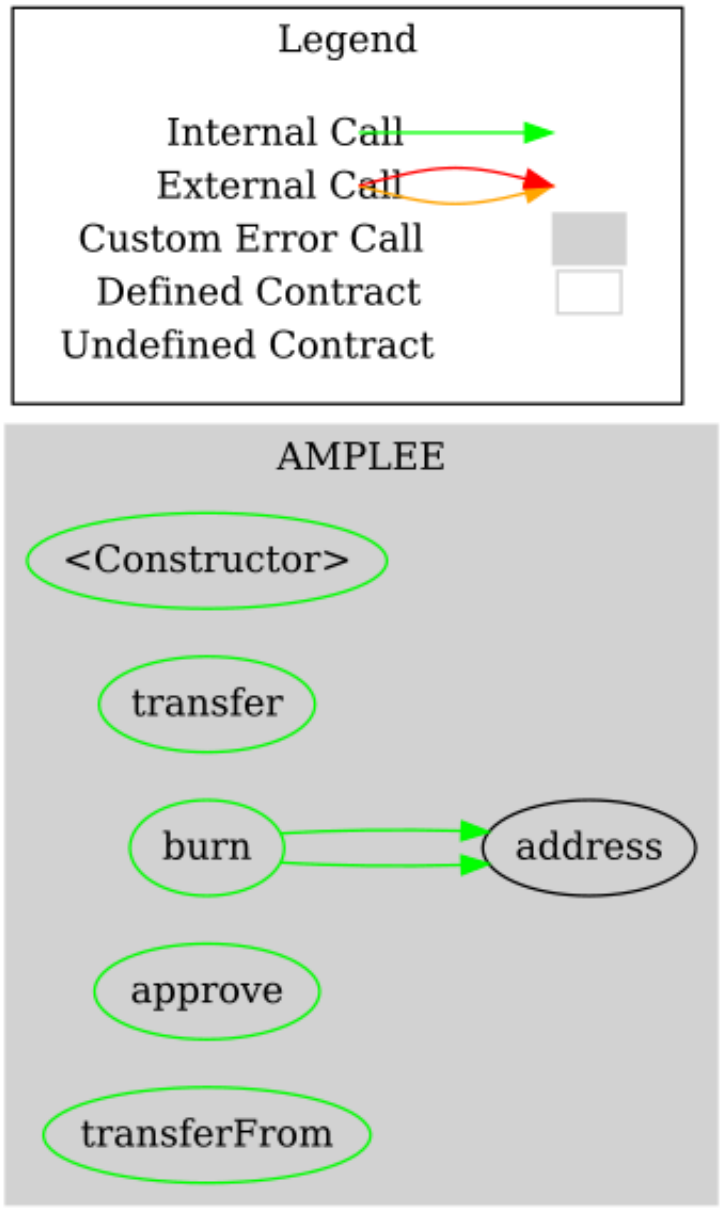
Functions Analysis

Contract	Type	Bases		
	Function Name	**Visibility**	**Mutability**	**Modifiers**
AMPLEE	Implementation			
		Public	✓	-
	transfer	Public	✓	-
	burn	Public	✓	-
	approve	Public	✓	-
	transferFrom	Public	✓	-

Inheritance Graph



Flow Graph


Legend
Internal Call
External Call
Custom Error Call
Defined Contract
Undefined Contract
AMPLEE
<Constructor>
transfer
burn
address
approve
transferFrom

Summary

AMPLE ECOSYSTEM contract implements a token mechanism. This audit investigates security issues, business logic concerns and potential improvements.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



A **TAC Security** Company

The Cyberscope team

cyberscope.io